

BFS – Breath First Search

Breadth-First Search (BFS) is a graph traversal algorithm that explores nodes level by level, starting from a selected source node. It uses a queue to ensure that all neighbors of a node are visited before moving to the next depth level. BFS works for both directed and undirected graphs and is especially useful for finding the shortest path in unweighted graphs because it reaches nodes using the minimum number of edges. During the traversal, BFS marks each visited node to avoid repeating work and continues until all reachable nodes are processed. This method is commonly used in problems such as checking connectivity, solving mazes, performing level-order traversal in trees, detecting connected components, and checking whether a graph is bipartite. Its time complexity is $O(V + E)$, since it processes every vertex and edge once, and its space complexity is $O(V)$ due to the queue and visited array.

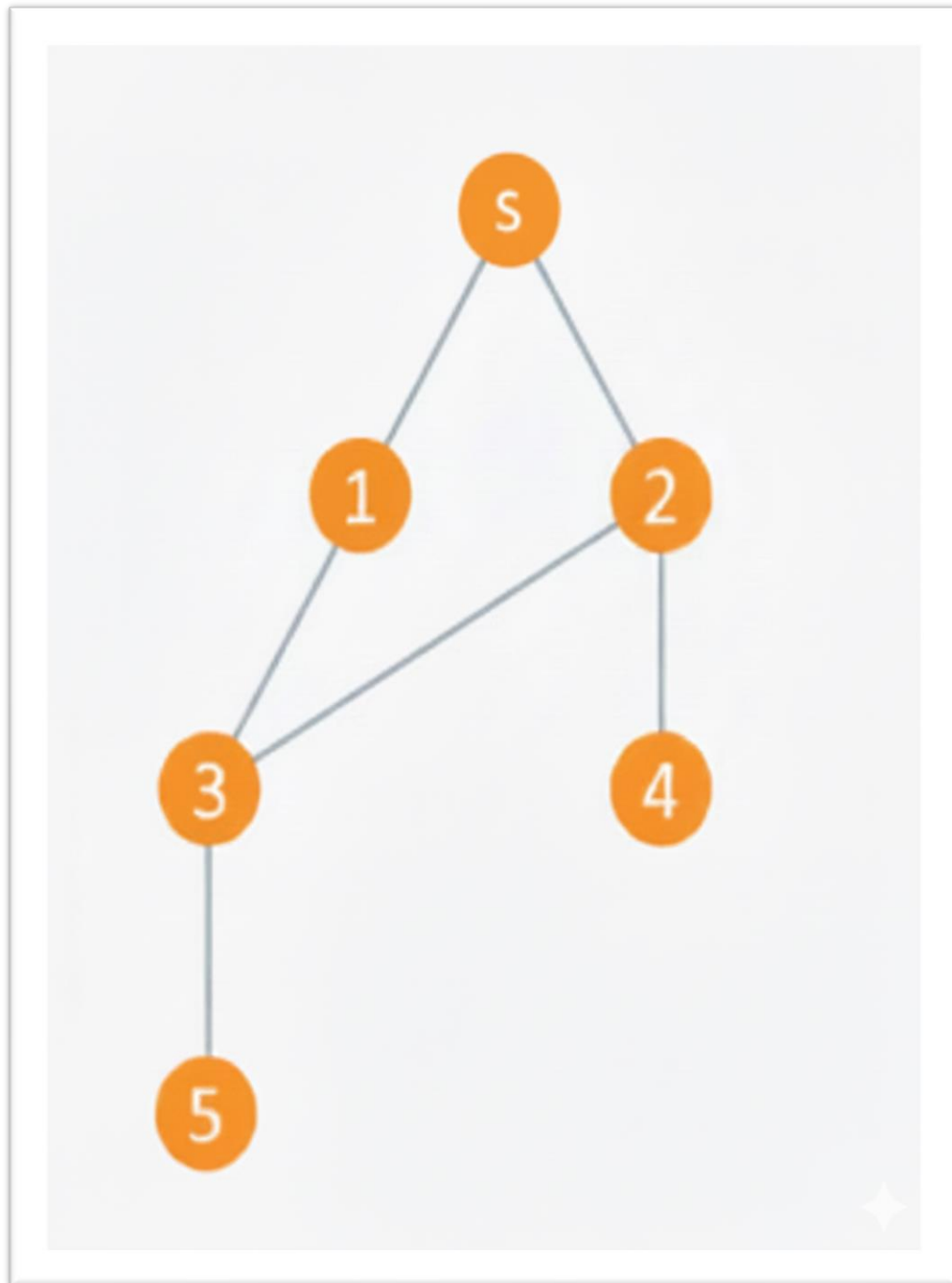
Advantages / Benefits of BFS

- BFS guarantees the shortest path in an unweighted graph because it explores nodes level by level.
- It is easy to implement and understand.
- Works well for checking connectivity and finding all reachable nodes from a source.
- Useful for solving problems on grids and trees, such as level-order traversal, shortest path in a maze, flood fill, and bipartite checking.
- BFS is systematic and complete, meaning it will eventually visit every reachable node.
- Reliable for exploring large search spaces where completeness is important.

Limitations of BFS

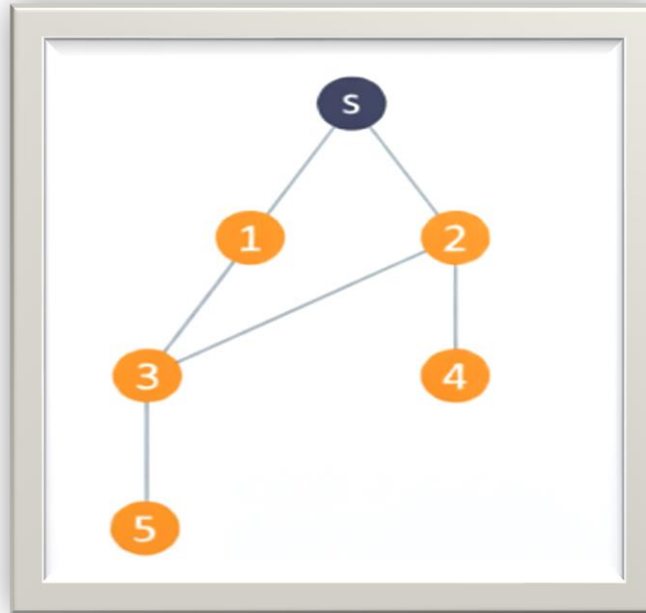
- BFS consumes more memory, since it stores all nodes of the current level in a queue.
- Can become inefficient in wide or dense graphs due to high space usage.
- BFS explores nodes blindly, without considering their importance or cost.
- Not suitable for weighted graphs unless combined with algorithms like Dijkstra's.
- In deep graphs, BFS can be slower than DFS, because it explores all shallow nodes first.
- May increase both time and space usage when the search depth is large.

Graph:



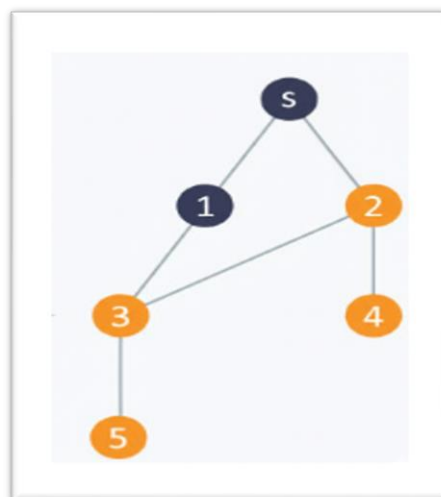
Steps For BFS:

Start:



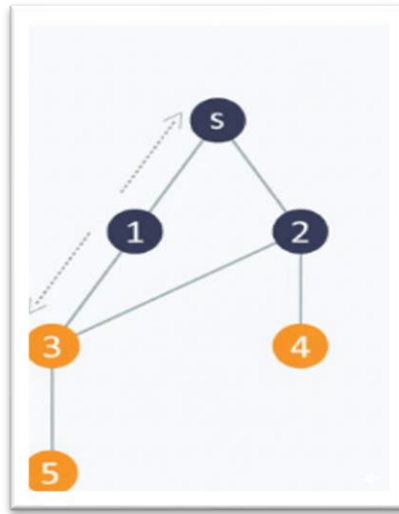
First iteration

- s will be popped from the queue
- Neighbors of s i.e. 1 and 2 will be traversed
- 1 and 2, which have not been traversed earlier, are traversed. They will be:
 - Pushed in the queue
 - 1 and 2 will be marked as visited



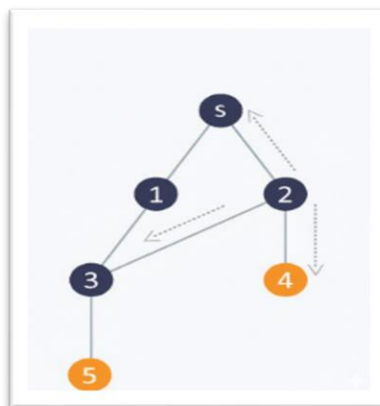
Second iteration

- 1 is popped from the queue
- Neighbors of 1 i.e. s and 3 are traversed
- s is ignored because it is marked as 'visited'
- 3, which has not been traversed earlier, is traversed. It is:
 - Pushed in the queue
 - Marked as visited



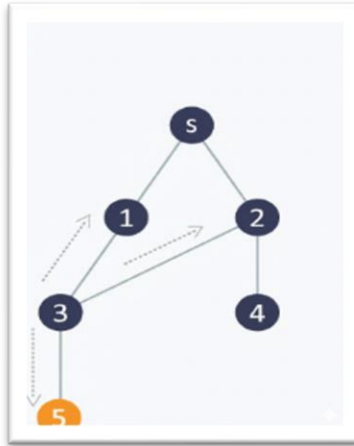
Third iteration

- 2 is popped from the queue
- Neighbors of 2 i.e. s, 3, and 4 are traversed
- 3 and s are ignored because they are marked as 'visited'
- 4, which has not been traversed earlier, is traversed. It is:
 - Pushed in the queue
 - Marked as visited



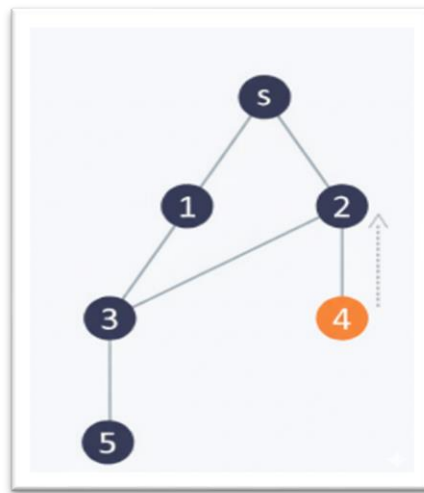
Fourth iteration

- 3 is popped from the queue
- Neighbors of 3 i.e. 1, 2, and 5 are traversed
- 1 and 2 are ignored because they are marked as 'visited'
- 5, which has not been traversed earlier, is traversed. It is:
 - Pushed in the queue
 - Marked as visited



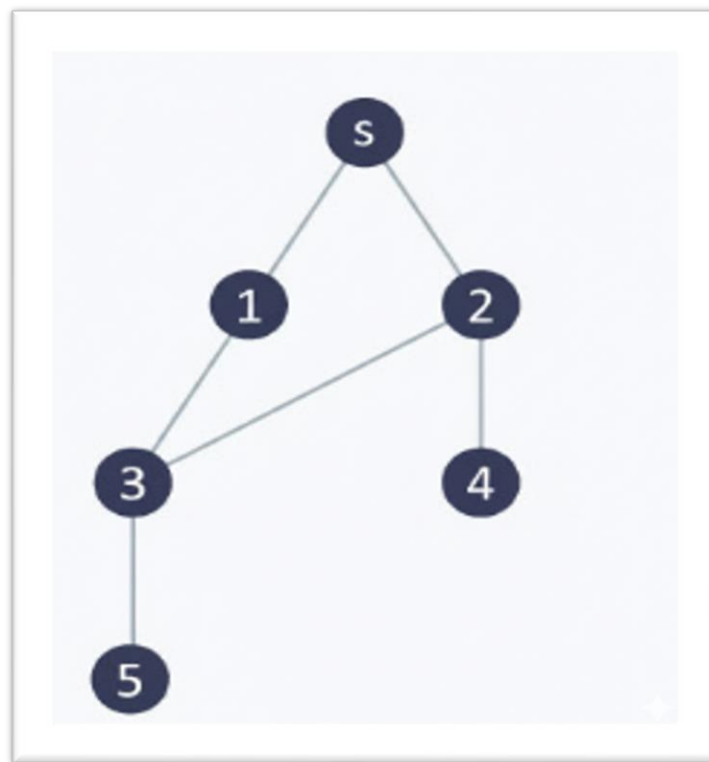
Fifth iteration

- 4 will be popped from the queue
- Neighbors of 4 i.e. 2 is traversed
- 2 is ignored because it is already marked as 'visited'



Sixth iteration

- 5 is popped from the queue
- Neighbors of 5 i.e. 3 is traversed
- 3 is ignored because it is already marked as 'visited'



The queue is empty and it comes out of the loop. All the nodes have been traversed by using BFS.

Pseudocode:

FUNCTION BFS(graph, source, n):

 FOR each vertex i from 0 to n-1:

 color[i] = WHITE

 distance[i] = INFINITY

 predecessor[i] = NIL

 color[source] = GRAY

 distance[source] = 0

 CREATE an empty queue Q

 ENQUEUE source into Q

 WHILE Q is not empty:

 u = DEQUEUE from Q

 FOR each vertex v in adjacency list of u:

 IF color[v] == WHITE:

 color[v] = GRAY

 distance[v] = distance[u] + 1

 predecessor[v] = u

 ENQUEUE v into Q

 color[u] = BLACK

Time Complexity:

The time complexity of BFS is:

$$O(V+E)$$

where:

- V = number of vertices (nodes)
- E = number of edges

Breakdown:

Operation	Description	Time Complexity
Initialization	Set color[], distance[], predecessor[] for all vertices	$O(V)$
Enqueue/Dequeue in Queue	Each vertex is added and removed exactly once	$O(V)$
Explore adjacency list	For each vertex, iterate through all its neighbors (sum over all vertices = E)	$O(E)$
Total Time Complexity	Sum of all above operations	$O(V + E)$

BFS Code:

<https://github.com/AHANIK33/Algorithm/blob/main/BFS/Basic/basic.cpp>